ReturnX – Smart Enterprise Lost & Found Management System

1. Document Information

- **Project Name:** ReturnX
- **Project Title:** ReturnX – Smart Enterprise Lost & Found Management System
- **Submission Date:** July 31, 2026
- **Document Purpose:** To provide a comprehensive technical and functional specification for the ReturnX platform as part of the GlobalLogic Java Track TE 2.0 Capstone Project.

2. Product Overview

Vision

To create a safe, open, and supportive workplace where lost items can be quickly returned to their owners using technology.

Goal

To deploy a production-ready, full-stack microservices application that automates lost and found management, eliminates administrative overhead, and incentivizes ethical employee behavior.

Business Context & Problem Statement

In large-scale enterprise settings like GlobalLogic, employees frequently misplace personal and corporate assets. Current recovery methods are fragmented, relying on manual HR intervention, mass emails, and disparate Slack/Teams channels, resulting in high noise and low recovery rates.

Existing Challenges

- Lack of a centralized, searchable repository for misplaced items.
- High administrative burden on HR and Facilities teams.

- Communication fatigue caused by organization-wide "lost item" blasts.
- No incentive or gamification to encourage reporting of found items.

Proposed Solution

ReturnX is an end-to-end cloud-native platform enabling employees to report lost/found items, search a global catalog, and engage in secure, real-time peer-to-peer chat for item verification and handover coordination.

Business Value

- **Operational Excellence:** Automated workflows reduce HR manual labor.
- **Rapid Recovery:** Direct communication accelerates the return cycle.
- **Cultural Engagement:** The Karma Point system promotes integrity and social responsibility.

3. Current Problem Analysis

The existing manual process in most enterprises suffers from significant bottlenecks:

1. **Manual HR Dependency:** Employees must physically surrender items, requiring staff to log inventory and manage storage.
2. **Communication Overload:** Broadcast messages on enterprise channels create noise and are easily missed by the intended recipient.
3. **Poor Visibility:** Claimants lack a searchable interface to verify if their item has been reported, leading to permanent loss of assets.
4. **Administrative Workload:** Coordinating meetups and verifying ownership manually consumes excessive productive time.
5. **Lack of Centralization:** Data tracked in disconnected spreadsheets prevents meaningful auditing or loss-prevention analysis.

4. Proposed Solution

ReturnX serves as the definitive single source of truth for all misplaced assets within the enterprise.

- **Platform Overview:** A distributed, microservices-based application featuring a responsive React frontend and a Spring Boot backend.

- **Core Capabilities:** Real-time search, automated claim management, secure Socket.IO chat, and a gamified reward engine.
- **Employee Experience:** Intuitive self-service portal for quick reporting and seamless peer-to-peer resolution without institutional red tape.
- **HR/Admin Benefits:** Transition from active management to passive oversight with robust analytics and dispute resolution tools.
- **Organizational Value:** Protects corporate assets (ID cards, laptops) while building trust and camaraderie among the workforce.

5. User Personas

Persona	Responsibilities	Goals	Pain Points	Primary Features
Claimant	Reports lost items; searches catalog.	Securely recover items quickly.	Anxiety; poor visibility; slow manual processes.	Report Lost, Search, Claim, Chat.
Finder	Reports found items; secures assets.	Efficiently return items; earn rewards.	Lack of time; fear of false claims.	Report Found, Chat, Handover, Leaderboard.
Admin	Oversees platform; resolves disputes.	Maintain system integrity; monitor data.	Wasted time on physical inventory.	Analytics, User Mgmt, Dispute Resolution.

6. Complete System Workflow

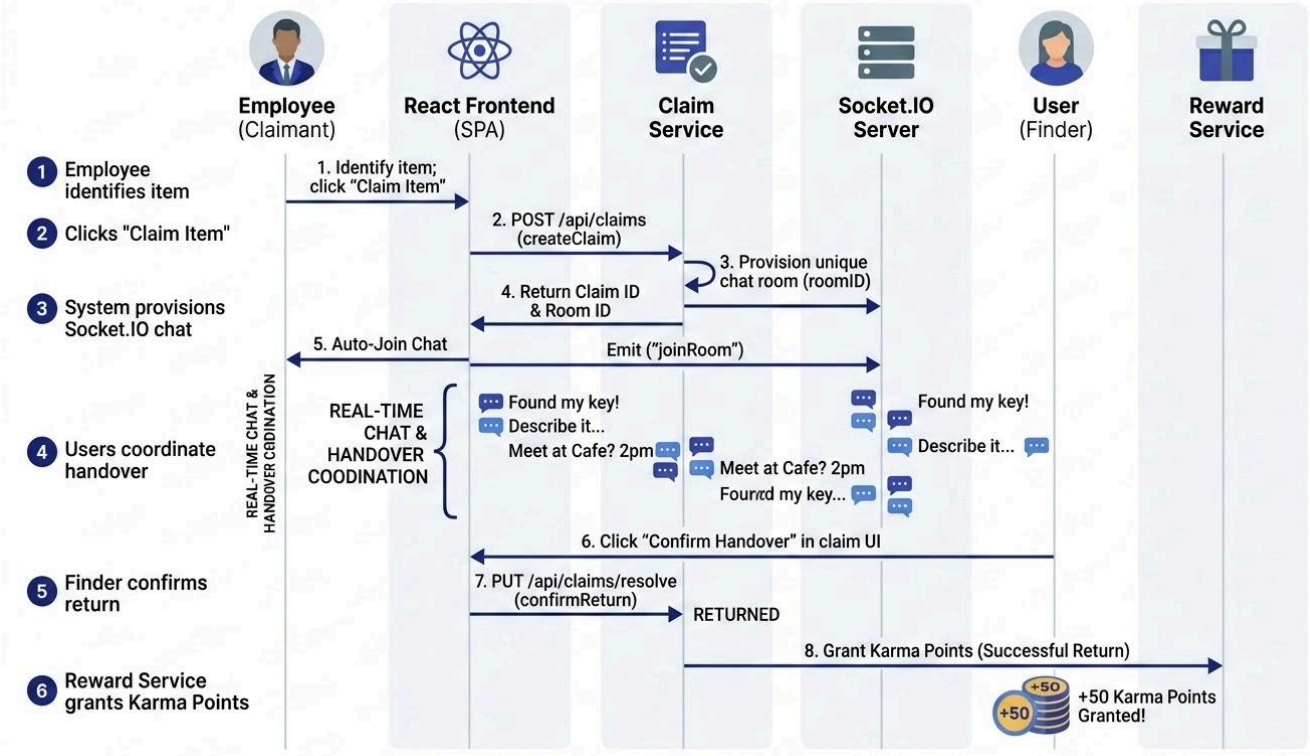

Figure 2: End-to-End Recovery and Reward Workflow

1. **Authentication:** Employee logs in via JWT-secured corporate credentials.
2. **Reporting:** User submits a "Lost Item" report or a "Found Item" report with images and location data.
3. **Discovery:** Items populate a centralized, searchable repository.
4. **Claim Initiation:** A claimant identifies a match and clicks "Claim Item."
5. **Auto-Chat Provisioning:** The Claim Service automatically generates a secure, dedicated chat room.
6. **Real-Time Communication:** Socket.IO session begins; users verify ownership through direct conversation.
7. **Handover Coordination:** Meeting time and location are finalized within the secure chat.
8. **Completion:** Finder confirms the physical handover; status updates to "RETURNED."
9. **Reward Distribution:** Reward Service automatically grants Karma Points to the Finder.
10. **Notifications:** Both users receive asynchronous alerts regarding claim status and point accrual.

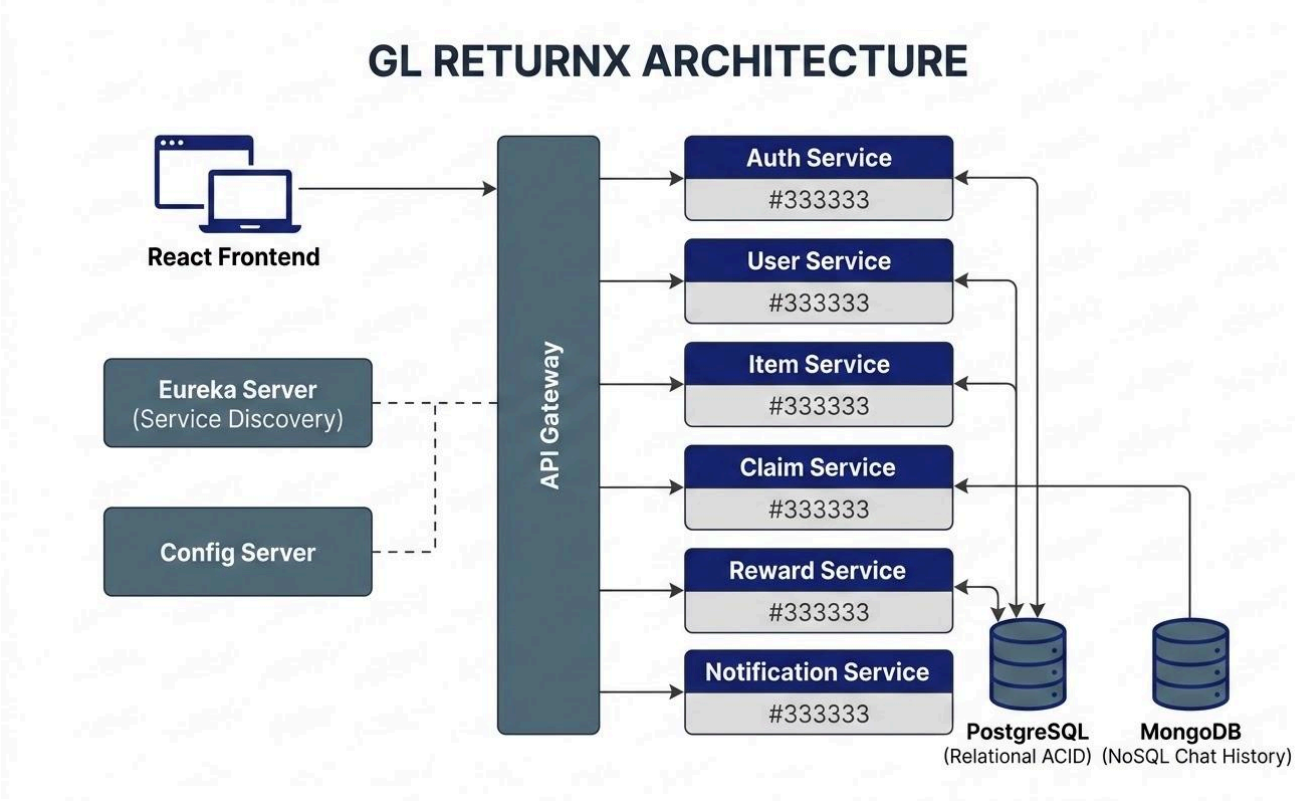

Figure 1: ReturnX Microservices Architecture

7. System Architecture

The architecture utilizes a distributed microservices model for high availability and independent scalability.

- **React Frontend:** Single Page Application (SPA) providing a responsive UX and Socket.IO client integration.
- **API Gateway:** The single entry point for all client requests; handles routing and cross-origin resource sharing (CORS).
- **Eureka Server:** Facilitates service discovery across the microservice ecosystem.
- **Config Server:** Centralized management for environmental configurations.
- **Auth Service:** Manages identity, BCrypt hashing, and JWT issuance.
- **User Service:** Maintains detailed employee profiles and preferences.
- **Item Service:** Core repository managing the lifecycle and search indices for lost and found assets.
- **Claim Service:** Orchestrates the recovery workflow and provisions real-time chat workspaces.
- **Reward Service:** Gamification engine for point calculation and leaderboard maintenance.

- **Notification Service:** Asynchronous event processor for in-app and email alerts.
- **Databases:** PostgreSQL (Relational/ACID for transactions) and MongoDB (NoSQL for chat history).

8. Technology Stack

Category	Technologies
Frontend	React, TypeScript, Tailwind CSS, shadcn/ui, Axios, Socket.IO Client
Backend	Java 17, Spring Boot 3.x, Spring Security, Spring Cloud, REST APIs, JWT, OpenFeign
Infrastructure	API Gateway, Eureka Server, Spring Cloud Config Server
Databases	PostgreSQL, MongoDB
Cloud	Vercel, Render, Neon PostgreSQL, Cloudinary
Testing	JUnit 5, Mockito, Test Driven Development (TDD)

9. Microservice Documentation

9.1 Auth Service

- **Purpose:** Identity provider and security gateway.
- **Responsibilities:** Registration, login, and JWT issuance/validation.
- **Major Features:** Password encryption, token-based state management.
- **Database:** PostgreSQL (Credentials).

9.2 User Service

- **Purpose:** Profile and metadata management.
- **Responsibilities:** CRUD operations for employee details and contact preferences.
- **Integration:** Consumed by Claim and Reward services for user identification.

9.3 Item Service

- **Purpose:** Asset lifecycle management.
- **Responsibilities:** Categorization, search, and image processing (Cloudinary).
- **Primary APIs:** `/api/items/lost`, `/api/items/found`, `/api/items/search`.

9.4 Claim Service

- **Purpose:** Transactional coordination.
- **Responsibilities:** Claim lifecycle tracking and automatic chat room generation.
- **Database:** PostgreSQL (Claim state), MongoDB (Chat transcripts).

9.5 Reward Service

- **Purpose:** Engagement and gamification.
- **Responsibilities:** Karma point assignment and global leaderboard updates.
- **Integration:** Triggered by the Claim Service upon successful item return.

9.6 Notification Service

- **Purpose:** Omnichannel user alerts.
- **Responsibilities:** Event-driven notifications for claims, chats, and rewards.
- **Primary APIs:** `/api/notifications`, `/api/notifications/read`.

10. Lost Item Workflow

- **Reporting:** Employees submit details including category, description, last seen location, and timestamp.
- **Business Validation:** Inputs are sanitized; date/time must be logical; description must meet minimum length requirements.
- **Search Availability:** Item enters the "LOST" state and is instantly indexed for global search.
- **Lifecycle:** LOST → CLAIM_PENDING → RETURNED.

11. Found Item Workflow

- **Reporting:** Finders upload images and provide found location and item condition.
- **Business Validation:** Mandatory image upload to facilitate visual matching by claimants.
- **Matching Process:** The system alerts users whose lost items share high-similarity attributes with the new report.

- **Lifecycle:** FOUND → CLAIM_PENDING → RETURNED.

12. Claim & Real-Time Chat Workflow

- **Claim Creation:** Initiated by a claimant upon identifying their item in the found catalog.
- **Chat Room Generation:** The system instantly provisions a private, secure Socket.IO room.
- **Socket.IO Communication:** High-concurrency, low-latency messaging for real-time interaction.
- **Verification:** Users conduct a mutual conversation to verify ownership (e.g., describing unique marks or contents).
- **Coordination:** Meeting details are finalized directly within the chat interface.
- **Completion:** Finder marks the item as "RETURNED" via the claim UI, finalizing the recovery.

13. Reward Management

The platform utilizes a gamified "Karma Points" system to incentivize participation.

Action	Karma Points	Condition
Report Found Item	+10 Points	Successful log in the system.
Successful Return	+50 Points	Handover confirmed by both parties.
First Return Milestone	+100 Points	One-time bonus for the first success.
Spam/False Claim	-30 Points	Deduction for administrative flags.

- **Leaderboard:** A dynamic dashboard ranking employees based on aggregate helpfulness.
- **Business Value:** Encourages proactive reporting and builds a culture of integrity.

14. Notification Module

- **Claim Alerts:** Instant notification when an ownership claim is placed on a found item.
- **Reward Alerts:** Confirmation messages when Karma Points are credited.
- **Matching Alerts:** Proactive notifications when a newly reported item matches a user's lost report.
- **Future Enhancements:** Native push notifications and MS Teams bot integration.

15. Database Architecture

ReturnX employs a **polyglot persistence strategy** to optimize for different data access patterns.

- **PostgreSQL:** Serves as the primary ACID-compliant store for transactional data (Users, Items, Claims, Rewards).
- **MongoDB:** Utilized for high-volume, unstructured chat transcripts and real-time message streams.
- **Loose Coupling:** Each microservice maintains its own logical schema, ensuring data independence.
- **Production Strategy:** Managed instances (Neon PostgreSQL and MongoDB Atlas) provide 99.9% availability and automated backups.

16. Security Requirements

- **JWT Authentication:** Stateless, token-based security for all API endpoints.
- **Role-Based Access Control (RBAC):** Distinct permissions for standard Employees and System Administrators.
- **BCrypt:** Strong cryptographic hashing for all credentials.
- **Input Validation:** Strict sanitization to mitigate XSS and SQL Injection risks.
- **Secure Communication:** Enforcement of HTTPS and WSS for all data in transit.

17. API Overview

Service	Primary Endpoints
Auth	`POST /login`, `POST /register`, `POST /validate`

Service	Primary Endpoints
User	`GET /profile`, `PUT /profile/update`
Item	`POST /items`, `GET /items/search`, `PUT /items/status`
Claim	`POST /claims`, `PUT /claims/resolve`
Reward	`GET /leaderboard`, `GET /history`

18. User Stories & Acceptance Criteria

US-001: Employee Registration/Login

- **Value:** Ensures secure access for corporate personnel.
- **Criteria:**
    - Given valid credentials, When the login form is submitted, Then a JWT is issued.
    - Given an invalid password, When submitted, Then access is denied and an error is shown.
    - Given a new employee, When registration is complete, Then a user profile is created.

US-002: Employee Profile Management

- **Value:** Keeps contact details current for handover coordination.
- **Criteria:**
    - Given a logged-in user, When they visit settings, Then their profile data is loaded.
    - Given updated details, When saved, Then the database updates successfully.
    - Given a profile update, When completed, Then a success notification is displayed.

US-003: Report Lost Item

- **Value:** Initiates the recovery lifecycle for missing assets.
- **Criteria:**
    - Given the lost item form, When all fields are valid, Then the item is saved as "LOST."
    - Given a missing description, When submitted, Then the system highlights the error.

    - Given a saved lost item, When viewed in the dashboard, Then its status is accurate.

US-004: Report Found Item

- **Value:** Enables finders to log assets for recovery.
- **Criteria:**
    - Given a found item form, When an image is uploaded, Then the item is saved as "FOUND."
    - Given a successful report, When processed, Then the finder receives 10 Karma Points.
    - Given an invalid image file, When uploaded, Then an error message is displayed.

US-005: Search & Filter Items

- **Value:** Accelerates item discovery via intuitive categorization.
- **Criteria:**
    - Given a keyword search, When submitted, Then matching items are displayed instantly.
    - Given a category filter, When applied, Then results are restricted to that category.
    - Given a date filter, When selected, Then only relevant chronological items appear.

US-006: Claim Item

- **Value:** Initiates the peer-to-peer verification phase.
- **Criteria:**
    - Given a found item, When the "Claim" button is clicked, Then a claim request is created.
    - Given an active claim, When viewed by others, Then the item status is "PENDING."
    - Given a successful claim creation, When finished, Then a notification is sent to the finder.

US-007: Automatic Chat Room Creation

- **Value:** Provides immediate, secure communication channels.
- **Criteria:**
    - Given a new claim, When saved, Then a unique Socket.IO room ID is generated.
    - Given the chat room, When generated, Then both parties are automatically joined.
    - Given the dashboard, When a claim is pending, Then a "Join Chat" button is visible.

US-008: Real-Time Chat using Socket.IO

- **Value:** Facilitates verification and coordination.
- **Criteria:**
    - Given an active room, When a message is sent, Then it is received instantly by the other user.
    - Given the chat UI, When a user types, Then a typing indicator is visible to the peer.
    - Given a returning user, When they re-open the chat, Then message history is loaded from MongoDB.

US-009: Complete Item Handover

- **Value:** Finalizes the recovery and updates system inventory.
- **Criteria:**
    - Given a physical handover, When the finder clicks "Confirm," Then status updates to "RETURNED."
    - Given a returned status, When processed, Then the item is removed from search results.
    - Given a resolved claim, When complete, Then the chat room becomes read-only.

US-010: Karma Points & Reward History

- **Value:** Motivates participation and recognizes integrity.
- **Criteria:**
    - Given a successful return, When finalized, Then 50 Karma Points are awarded.
    - Given the leaderboard, When viewed, Then users are ranked by aggregate points.
    - Given the user profile, When accessed, Then a transaction history of points is visible.

19. Non-Functional Requirements

- **Security:** 100% endpoint coverage with JWT; BCrypt hashing for sensitive data.
- **Scalability:** Microservices architecture supporting horizontal scaling via API Gateway.
- **Reliability:** Service discovery via Eureka ensures fault tolerance and rerouting.
- **Availability:** 99.9% uptime target via managed cloud databases (Neon).
- **Performance:** API latency < 200ms; WebSocket message latency < 50ms.
- **Maintainability:** Strict adherence to TDD and clean code principles.

20. Future Enhancements

- **AI Image Matching:** Auto-suggestions based on computer vision comparison.
- **OCR Integration:** Scanning ID badges for automated owner identification.
- **Mobile App:** Dedicated iOS/Android versions built with React Native.
- **MS Teams Integration:** In-chat notifications and command triggers within the workspace.
- **Event-Driven Architecture:** Implementing Kafka for robust asynchronous communication.
- **Analytics Dashboard:** HR heatmaps showing frequent loss locations.

21. Conclusion

ReturnX represents a modernized, efficient, and collaborative approach to enterprise asset recovery. By leveraging a robust Spring Boot microservices backend and a high-performance React frontend, the platform eliminates legacy manual bottlenecks. The integration of Socket.IO facilitates a self-governed recovery workflow through real-time communication, fostering a corporate culture of integrity and social responsibility.

“Every Lost Item Deserves a Safe Return.”